

PROGRAMMABLE SMART-FLUID MATRIX

Complete Prototype, Wiring, Firmware, Python Control, Testing, Safety, and Commercialization
Guide

Prepared for: Michael Glivar

Project: Electrically addressable particle-suspension research platform

Revision: Prototype planning edition — July 2026

Important: This document describes an experimental low-voltage research prototype. It does not establish that the device is intelligent, commercially ready, or safe for unattended operation. Start with graphite, not carbon nanotubes; use a fused, current-limited supply; and keep the chamber covered.

1. Project Purpose

Build a programmable chamber containing electrically responsive particles suspended in a dielectric fluid. A computer sends commands to a microcontroller, which controls an electrode array intended to move, align, concentrate, and potentially connect particles into temporary, measurable patterns.

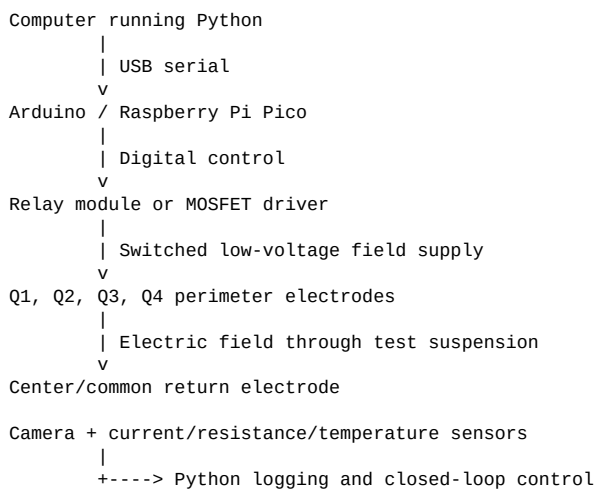
Primary objective: demonstrate repeatable, reversible, spatially addressable particle movement. **Secondary objective:** determine whether those structures can form measurable conductive pathways. **Advanced objective:** add sensor and camera feedback so software can optimize the material state rather than merely switch electrodes randomly.

2. What Success Means

- Field-controlled migration: voltage produces a particle response distinguishable from an oil-only control.
- Spatial addressability: different electrodes produce measurably different responses.
- Reversible assembly: structures form under a field and substantially disperse or change after removal.
- Repeatability: the same command produces similar results over many cycles.
- Conductive switching: activated structures significantly reduce resistance between selected terminals.
- Closed-loop adaptation: sensor measurements determine subsequent electrode commands.

The prototype alone does not prove consciousness, autonomous intelligence, self-repair, learning, or a new state of matter. Random relay activation is open-loop sequencing, not learning.

3. Recommended Architecture



The four switched electrodes require an opposing return electrode. Four positive quadrants alone do not establish useful quadrant-to-quadrant field paths. Start with one common center electrode connected to the external supply return.

4. Bill of Materials

Chamber and suspension

- 60–90 mm glass Petri dish with clear cover.
- Flat nonconductive mounting base and secondary spill tray.
- 100 mL light mineral oil or silicone dielectric oil.
- Fine lubricant-grade graphite powder; do not begin with loose MWCNT powder.

- 0.01 g-resolution scale, graduated syringe, glass stirring rod, pipettes, and labeled sample containers.
- Nitrile gloves, safety glasses, particulate control, and disposable bench liner.

Electronics

- Arduino Uno R3, Nano, or Raspberry Pi Pico with data cable.
- Four-channel relay board or, preferably, a four-channel logic-level MOSFET driver.
- Solderless breadboard, jumper wires, hook-up wire, terminal blocks, and heat-shrink tubing.
- Copper foil, copper PCB, stainless-steel electrodes, or graphite electrodes for geometry testing.
- Adjustable current-limited DC bench supply, inline fuse, master switch, and digital multimeter.
- Optional current sensor, temperature sensor, ADC, USB oscilloscope, and overhead camera.

5. Safety Requirements

1. Use a fused, current-limited, low-voltage supply. Do not connect mains power to the chamber.
2. Start at 3–5 V and raise voltage only in small steps while monitoring current and temperature.
3. Keep the chamber covered and place it in secondary containment.
4. Avoid airborne graphite. Do not use loose carbon-nanotube powder without professional exposure controls.
5. Disconnect power before changing electrodes or wiring.
6. Stop immediately for heating, bubbling, corrosion, arcing, smoke, odor, or rapidly rising current.
7. Keep Arduino USB circuitry isolated from the electrode supply unless the selected driver specifically requires a shared reference.
8. Do not leave the prototype operating unattended.

6. Electrode Geometry

Configuration A — electrodes beneath glass: cleaner and easier to seal, but steady DC may couple weakly through insulating glass. AC or pulsed fields may eventually be needed.

Configuration B — electrodes contacting fluid: stronger interaction and direct resistance measurement, but introduces corrosion, contamination, leakage, and possible electrochemical effects. Use replaceable electrodes and conservative current limits.

Recommended first geometry: four perimeter electrodes labeled Q1–Q4 and one center return electrode. Use consistent spacing and document every dimension. Prevent direct metal-to-metal contact and accidental shorts.

7. Relay Wiring

- Arduino 5 V → relay VCC.
- Arduino GND → relay GND.
- Arduino D2 → IN1; D3 → IN2; D4 → IN3; D5 → IN4.
- Fused external positive → COM1, COM2, COM3, and COM4.
- NO1 → Q1; NO2 → Q2; NO3 → Q3; NO4 → Q4.
- External supply negative → center/common return electrode.
- Do not automatically connect external negative to Arduino ground; relay contacts normally isolate the circuits.

Critical: verify whether the relay module is active-low. Many common modules energize when the input is LOW.

8. Arduino Firmware

Create a new Arduino sketch, paste the complete code below, select the correct board and port, and upload. The comments inside the code explain configuration and operation.

```
/*
  PROGRAMMABLE SMART-FLUID MATRIX
  Four-channel serial electrode controller

  SAFETY:
  1. Test relay logic before connecting the external field supply.
  2. Confirm every channel is OFF during boot and reset.
  3. Use a fused, current-limited, low-voltage supply.
  4. Press/send X to turn every channel off.
*/

const byte RELAYS[] = {2, 3, 4, 5};
const byte RELAY_COUNT = 4;

// Many relay boards are active-low. If testing shows yours is
// active-high, change true to false and upload again.
const bool ACTIVE_LOW = true;

byte relayOnLevel() {
  return ACTIVE_LOW ? LOW : HIGH;
}

byte relayOffLevel() {
  return ACTIVE_LOW ? HIGH : LOW;
}

void setChannel(byte channel, bool enabled) {
  if (channel >= RELAY_COUNT) return;
  digitalWrite(
    RELAYS[channel],
    enabled ? relayOnLevel() : relayOffLevel()
  );
}

void alloff() {
  for (byte i = 0; i < RELAY_COUNT; i++) {
    setChannel(i, false);
  }
}

void setup() {
  Serial.begin(9600);

  // Write the inactive level before switching each pin to OUTPUT.
  // This reduces unwanted relay pulses during startup.
  for (byte i = 0; i < RELAY_COUNT; i++) {
    digitalWrite(RELAYS[i], relayOffLevel());
    pinMode(RELAYS[i], OUTPUT);
  }

  alloff();
  Serial.println("SMART_FLUID_CONTROLLER_READY");
}

void loop() {
  if (Serial.available() <= 0) return;

  char command = Serial.read();

  switch (command) {
    case '1': setChannel(0, true); break; // Q1 ON
    case 'A': setChannel(0, false); break; // Q1 OFF
    case '2': setChannel(1, true); break; // Q2 ON
    case 'B': setChannel(1, false); break; // Q2 OFF
    case '3': setChannel(2, true); break; // Q3 ON
    case 'C': setChannel(2, false); break; // Q3 OFF
    case '4': setChannel(3, true); break; // Q4 ON
    case 'D': setChannel(3, false); break; // Q4 OFF
    case 'X': alloff(); break; // Emergency global OFF
    case '?': Serial.println("READY"); break;
  }
}
```

```
}  
}
```

9. Relay Verification

1. Leave the external supply disconnected.
2. Upload the firmware and power the Arduino only through USB.
3. Confirm all relays remain off at boot.
4. Use the Serial Monitor to send 1, A, 2, B, 3, C, 4, D, and X.
5. Confirm only the intended channel responds.
6. Disconnect and reconnect USB and verify a safe all-off restart.
7. If behavior is reversed, change `ACTIVE_LOW` and retest.
8. Do not attach the electrode supply until all startup and shutdown tests pass.

10. Fluid Preparation

Prepare multiple small samples rather than one large batch. Use the same oil volume, depth, mixing time, settling time, and chamber geometry for each test.

- Sample A: oil only—the mandatory control.
 - Sample B: 0.5% graphite by mass.
 - Sample C: 1% graphite by mass.
 - Sample D: 2% graphite by mass.
 - Sample E: 5% graphite by mass.
1. Clean and completely dry the chamber and tools.
 2. Measure oil mass and volume.
 3. Weigh graphite with dust controls in place.
 4. Add graphite slowly and stir for 3–5 minutes.
 5. Label the sample with formulation, date, oil, graphite grade, and particle size.
 6. Allow a defined settling period and photograph the sample.
 7. Add a shallow, repeatable fluid layer to the chamber and record its depth.

11. Python Controller

Save this as **smart_fluid_control.py**. The comments inside the script provide setup, operating, logging, and shutdown instructions.

```
"""  
PROGRAMMABLE SMART-FLUID MATRIX — HOST CONTROLLER  
  
SETUP  
1. Install Python 3.  
2. Install PySerial:  
    python3 -m pip install pyserial  
3. On macOS, list serial ports:  
    ls /dev/cu.*  
4. Replace SERIAL_PORT below with the Arduino port.  
5. Upload the Arduino firmware before running this file.  
6. Verify relay behavior with the external field supply disconnected.  
7. Run:  
    python3 smart_fluid_control.py
```

8. Press Ctrl+C for a controlled shutdown.

The program first runs a repeatable state sequence, then random states. Random switching is an experiment mode—not closed-loop learning.

```
"""

import atexit
import csv
import random
import signal
import time
from datetime import datetime

import serial

SERIAL_PORT = "/dev/cu.usbmodem1101" # CHANGE THIS
BAUD_RATE = 9600
STEP_SECONDS = 1.5
ACTIVATION_PROBABILITY = 0.30
LOG_FILE = "smart_fluid_test_log.csv"

arduino = None
running = True

def connect():
    """Open the serial connection and force all channels off."""
    global arduino
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2) # Arduino commonly resets when serial opens.
    arduino.reset_input_buffer()
    arduino.write(b"X")
    arduino.flush()

def send(command):
    """Send one ASCII command to the microcontroller."""
    if arduino and arduino.is_open:
        arduino.write(command.encode("ascii"))
        arduino.flush()

def all_off():
    """Request immediate deactivation of every electrode."""
    try:
        send("X")
    except Exception:
        pass

def shutdown():
    """Turn everything off and close the serial port."""
    global arduino
    all_off()
    if arduino and arduino.is_open:
        arduino.close()

def stop_handler(signum, frame):
    """Handle Ctrl+C or operating-system termination."""
    global running
    running = False

def initialize_log():
    """Create the CSV header only when the file is new."""
    try:
        with open(LOG_FILE, "x", newline="") as file:
            writer = csv.writer(file)
            writer.writerow([
                "timestamp", "q1", "q2", "q3", "q4", "mode"
            ])
    except FileExistsError:
        pass

def log_state(state, mode):
    """Record every commanded state for later video/data matching."""
    with open(LOG_FILE, "a", newline="") as file:
        writer = csv.writer(file)
```

```

        writer.writerow([
            datetime.now().isoformat(),
            int(state[0]), int(state[1]),
            int(state[2]), int(state[3]),
            mode,
        ])

def apply_state(state):
    """Apply [Q1,Q2,Q3,Q4] Boolean state to the Arduino."""
    on_commands = ["1", "2", "3", "4"]
    off_commands = ["A", "B", "C", "D"]
    for index, enabled in enumerate(state):
        send(on_commands[index] if enabled else off_commands[index])

def random_state():
    """Generate one open-loop random test state."""
    return [
        random.random() < ACTIVATION_PROBABILITY
        for _ in range(4)
    ]

def sequence_test():
    """Run a repeatable sequence before any random testing."""
    states = [
        [False, False, False, False],
        [True, False, False, False],
        [False, True, False, False],
        [False, False, True, False],
        [False, False, False, True],
        [True, True, False, False],
        [False, False, True, True],
        [True, False, True, False],
        [False, True, False, True],
        [True, True, True, True],
        [False, False, False, False],
    ]

    for state in states:
        if not running:
            break
        apply_state(state)
        log_state(state, "sequence")
        print(f"Applied state: {state}")
        time.sleep(STEP_SECONDS)

def random_test():
    """Run open-loop random patterns until the user stops the program."""
    while running:
        state = random_state()
        apply_state(state)
        log_state(state, "random")
        print(f"Applied state: {state}")
        time.sleep(STEP_SECONDS)

if __name__ == "__main__":
    atexit.register(shutdown)
    signal.signal(signal.SIGINT, stop_handler)
    signal.signal(signal.SIGTERM, stop_handler)
    initialize_log()

    try:
        connect()
        print("Smart-fluid controller connected.")
        print("Running repeatable sequence test.")
        sequence_test()

        if running:
            print("Running random pattern test.")
            random_test()

    except Exception as error:
        print(f"Controller error: {error}")

    finally:
        shutdown()

```

```
print("All electrodes off. Serial connection closed.")
```

12. Computer Commands

```
# Install the serial library
python3 -m pip install pyserial

# macOS: list likely serial devices
ls /dev/cu.*

# Run the controller
python3 smart_fluid_control.py

# Stop safely
Press Ctrl+C
```

13. Initial Test Procedure

1. Place the device in secondary containment and install the oil-only control.
2. With external power off, connect the controller and verify relay status.
3. Set the bench supply to 3–5 V with a conservative current limit.
4. Turn on the external supply and activate one electrode at a time.
5. Record voltage, current, temperature, command, and visible response.
6. Repeat the same sequence at least ten times.
7. Turn off and disconnect field power before replacing the sample.
8. Test the lowest graphite concentration using the identical procedure.
9. Increase concentration before increasing voltage.
10. Save video and logs under a common test identification number.

14. Required Test Record

- Test ID, date, time, chamber revision, and operator.
- Electrode material, dimensions, spacing, and contact/non-contact arrangement.
- Oil type, graphite source and grade, concentration, mixing time, settling time, and fluid depth.
- Supply voltage, current limit, measured current, activation time, and commanded state.
- Formation time, dissolution time, resistance before/during/after activation, and temperature.
- Video/image filenames, observations, abnormalities, and pass/fail result.

15. Stage-Gate Criteria

Stage 1 — visible response

- Movement occurs under the applied field and differs from the oil-only control.
- Electrodes produce location-specific responses.
- The response repeats at least ten times.

Stage 2 — reversible structure

- A defined structure forms, changes after deactivation, and has measurable formation/dissolution times.
- Behavior remains acceptably repeatable for at least 50 cycles.

Stage 3 — conductive switching

- Resistance decreases significantly during activation and trends back toward baseline afterward.
- The change is repeatable and is not caused by a direct short, moisture, or electrode contamination.

Stage 4 — programmable routing

- Different commands create electrically distinct paths and the desired path is software selectable.

Stage 5 — closed-loop adaptation

- Sensors measure the outcome, software scores it, and the next electrode state is selected from feedback.
- The measured objective improves beyond an appropriate random-control baseline.

16. Camera Feedback Plan

1. Rigidly mount a camera above the chamber with fixed lighting, focus, exposure, background, and chamber position.
2. Capture a baseline image with every electrode off.
3. Apply one electrode state and wait a fixed interval.
4. Capture a response image and compare it with baseline.
5. Measure dark-particle density, pattern location, bridge continuity, or similarity to a target.
6. Log the command, image, current, resistance, temperature, and score.
7. Select the next state using the score rather than a random number.

A useful optimization target is: find the lowest-resistance path between two terminals while minimizing power, temperature rise, and unintended connections.

17. Commercial Direction

The shortest path to a product is likely a sealed programmable smart-fluid STEM or museum-display kit. The more technically valuable—but harder—direction is a repeatable reconfigurable conductive-fluid switch or research platform.

- Educational kit: sealed chamber, controller, application, guided experiments, and data logging.
- Interactive display: sound-, motion-, or sensor-responsive particle patterns.
- Research platform: interchangeable chambers, electrode geometries, measurement channels, and analysis software.
- Conductive device: temporary routing, resettable switching, adaptive sensors, or unconventional-computing research.

A buyer will require a specific function, repeatability data, safe stable materials, shelf-life evidence, manufacturing consistency, unit economics, competitive comparison, and a defensible intellectual-property position.

18. Intellectual Property

1. Maintain a dated inventor notebook and source-control repository.
2. Save every formulation, geometry, data file, image, and video.
3. Search prior art covering electrorheology, dielectrophoresis, colloidal assembly, conductive suspensions, liquid circuits, and reconfigurable electronics.
4. Identify the novel combination—not merely known particle chaining.
5. Consider a provisional patent filing before broadly disclosing novel formulas, geometry, feedback methods, or demonstrated performance.

6. Potential value may lie in chamber geometry, electrode arrangement, suspension formulation, measurement method, calibration, feedback algorithm, or a specific industrial application.

19. Project Statement

I am developing a programmable smart-fluid matrix that uses software-controlled electric fields to organize suspended particles into temporary, measurable, and potentially conductive structures. The first objective is repeatable, reversible, spatially addressable particle movement. The second is programmable conductive pathways. The third is closed-loop sensor feedback that automatically optimizes the material state. Depending on measured performance, the technology could become an educational kit, interactive display, research platform, adaptive sensor, or reconfigurable fluid-circuit system.

20. Immediate Milestones

1. One command creates one repeatable response in a known location, and the response changes when the command is removed.
2. The particle structure creates a repeatable, measurable resistance change.
3. Sensor-driven software produces a better measured result than random switching.

21. Technical Notes

Mechanical relays are acceptable for slow testing. MOSFET drivers are preferable for high-cycle operation, pulse-width control, waveform experiments, and silent switching. Any redesign that applies both polarities or AC requires an appropriate H-bridge or isolated driver; do not improvise polarity reversal with incompatible hardware.

This is an experimental plan rather than a guarantee of visible bridging at 12 V. Particle chemistry, concentration, electrode spacing, waveform, fluid depth, conductivity, moisture, and field strength will determine the result.

22. Reference

ReportLab was used to generate this PDF. Its documentation describes programmatic PDF generation through Python. [web:58]